Assignment NLU Report

Jayant Parmar
Postgraduate Diploma in Artificial Intelligence
Indian Institute of Technology Jodhpur, Jodhpur, India
Email: g25ait1072@iitj.ac.in

Deeksha Varshney

School of Artificial Intelligence & Data Science (SAIDE)
Indian Institute of Technology Jodhpur, Jodhpur, India
Email: deeksha@iitj.ac.in

Introduction

This report documents a baseline Sanskrit-to-English neural machine translation solution built from the provided dataset in the Data_to_Students folder. The solution uses a custom sequence-to-sequence architecture with an encoder, decoder, and attention mechanism.

Architecture

- **Encoder:** LSTM-based encoder over Sanskrit token sequences.
- **Decoder:** LSTM-based decoder that generates English tokens step by step.
- **Attention:** Dot-product attention over encoder outputs to help the decoder focus on relevant source-side hidden states.
- **Beam Search:** The submitted baseline uses greedy decoding for reproducibility and speed. Beam search was considered as an extension because it can keep multiple candidate translations at each decoding step, but it was not enabled in the final reported run.

Training Procedure

- **Preprocessing:** The dataset was split into training and development portions, and each sentence pair was cleaned using Unicode-aware handling before vocabulary construction.
- **Tokenization:** Whitespace and punctuation tokenization was applied using Unicode-aware regular expressions so that Sanskrit and English tokens could be represented consistently.
- **Vocabulary:** Source and target vocabularies were built from the training and development splits, including special tokens for padding, unknown words, sentence start, and sentence end.
- **Optimization:** The model was optimized using the Adam optimizer with cross-entropy loss.
- **Regularization:** The solution relied mainly on the compact model size, limited hidden dimension, and controlled number of epochs to reduce overfitting. No additional dropout-based regularization was used in the final reported baseline.

- **Hyperparameters:** Embedding size 128, hidden size 128, batch size 64, 25 epochs.
- **Pre-trained model disclosure:** DistilBERT-base-uncased was used only for BERTScore evaluation. The translation model itself was trained from scratch on the provided dataset.

Results

Metric	Development Split Result
BLEU	0.08059413235176767
BERTScore F1	0.769096314907074
Inference Time	13.29 seconds
Total Model Parameters	11,785,072

The model was trained on CPU for 25 epochs. Training loss decreased from an average loss of 5.1468 in epoch 1 to 1.6459 in epoch 25. The final development results were BLEU 0.08059413235176767 and BERTScore F1 0.769096314907074. Inference time was 13.29 seconds, and the model contained 11,785,072 trainable parameters. The generated predictions were saved to submission.csv.

Efficiency on Development and Test Data

Development-set efficiency was measured using the reported inference time of 13.29 seconds and the total parameter count of 11,785,072. Test-set efficiency was not separately reported in the available run output; the same inference pipeline is used for generating the submission.csv file.

Training Log Summary

Epoch	Average Loss
1	5.1468
5	3.6521
10	2.7744
15	2.2195
20	1.8789
25	1.6459

The training-loss curve shows a consistent downward trend across the selected epochs, decreasing from 5.1468 at epoch 1 to 1.6459 at epoch 25. This indicates that the model learned progressively during training, although translation quality remained limited because the architecture and dataset setup were intentionally lightweight.

Translation Examples

The notebook generates translation examples and saves predictions in the submission file. The table below should be populated with representative examples from the generated output.

No.	Source Sanskrit	Reference English	Model Prediction	Error Analysis
1	To be filled from notebook output	To be filled from dataset	To be filled from submission.csv	Check lexical accuracy, word order, and omitted words.
2	To be filled from notebook output	To be filled from dataset	To be filled from submission.csv	Check whether the main meaning is preserved.

3	To be filled from notebook output	To be filled from dataset	To be filled from submission.csv	Identify mistranslated or unknown tokens.
4	To be filled from notebook output	To be filled from dataset	To be filled from submission.csv	Check fluency and grammatical correctness.
5	To be filled from notebook output	To be filled from dataset	To be filled from submission.csv	Compare semantic similarity with the reference sentence.

Experiments and Discussion

The main experiment trained a lightweight LSTM encoder-decoder model with dot-product attention for 25 epochs on CPU. The design prioritized reproducibility, simplicity, and assignment feasibility over state-of-the-art performance. Greedy decoding was selected because it is deterministic and fast, while beam search was left as a future enhancement.

Design Choices

The encoder-decoder architecture was chosen because it provides a clear baseline for sequence-to-sequence translation. LSTM layers were used to model token dependencies, attention was included to improve alignment between Sanskrit input tokens and English output tokens, and compact embedding and hidden sizes were selected to keep training practical on CPU.

Challenges

Key challenges included limited training data, Sanskrit morphological variation, vocabulary sparsity, long-distance dependencies, and the difficulty of producing fluent English translations from a lightweight model. CPU-only training also constrained the number of experiments that could be run.

Limitations and Future Work

The model remains a baseline system and is not expected to match modern transformer-based translation systems. Future improvements could include subword tokenization, larger hidden dimensions, dropout regularization, longer training with validation monitoring, beam search decoding, and use of stronger pre-trained multilingual models for translation rather than evaluation only.

References

- NLTK BLEU score documentation.
- BERTScore official library.
- PyTorch documentation.